

```

•     ename = models.CharField(max_length=100)
•     econtact = models.CharField(max_length=15)
•     class Meta:
•         db_table = "employee"

```

Now, we would be creating the form using the above model.

```

• from django import forms
• from myapp.models import Employee
•
• class EmployeeForm(forms.ModelForm):
•     class Meta:
•         model = Employee
•         fields = "__all__"

```

This completes our form. We would now add the code for data validation using the `is_valid()` function.

```

• def emp(request):
•     if request.method == "POST":
•         form = EmployeeForm(request.POST)
•         if form.is_valid():
•             try:
•                 return redirect('/')
•             except:
•                 pass
•     else:
•         form = EmployeeForm()
•     return render(request, 'index.html', {'form': form})

```

Thus, we have added the form validation method for our form. Now, we have to ensure that the form is rendered using the index templates. The form can now be rendered, where the user types in the data into the fields. Any junk data throws an error as shown in the screenshot on the next page.

Uploading a File to a Form in Django

We have seen many examples of forms where the user can upload a file/document to the server. This feature makes it easier for the database administrators to collect relevant files from the users. Django allows file uploads to the server and makes this an easy task. There are multiple built-in functions and methods in Django to create a form that allows you to upload a file.